Código-fonte

Aluna

Vanessa Byork Ferreira Pinto

README.md

```
# Sistema de Controle de Estoque

Projeto desenvolvido como Produção Individual Textual (PTI) do curso de Práticas
de Programação no SENAC.

O sistema tem como objetivo gerenciar produtos em estoque, permitindo cadastro e
listagem de quantidades através de interface console.

## Objetivo do Projeto

Desenvolver um sistema para controle de estoque que permita:

- Cadastro de produtos
- Controle de quantidade
- Visualização de estoque

O projeto aplica conceitos de:

- Programação orientada a objetos
- Estrutura de dados
- Manipulação de arquivos CSV
- Injeção de dependência (Repository + Service)

## Tecnologias Utilizadas

- Python

## Estrutura do Projeto

O projeto segue uma separação por camadas:

```
controle_estoque
├── data
│   └── produtos.csv
├── src
│   └── app
│   ├── config.py
│   ├── main.py
│   ├── models
│   │   └── produto.py
│   ├── repositories
```
```

```
│       │   ├── memoria_produtos.py
│       │   ├── produtos.py
│       │   └── protocols.py
│       ├── services
│       │   └── produtos.py
│       └── utils
│           ├── arquivos.py
│           ├── console.py
└── pyproject.toml
```

Camadas:

- Models: Representação das entidades
- Repositories: Manipulação de dados
- Services: Regras de negócio
- Utils: Funções auxiliares
- App: Ponto de entrada da aplicação

Como Executar o Projeto

1. Clone o repositório:

```
   git clone https://github.com/Byorks/PTI-2S-SENAC-PP.git
```

2. Acesse a pasta do projeto:

```
   cd PTI-2S-SENAC-PP
```

3. Execute o projeto
   Na raiz do projeto execute:

```
   cd controle_estoque/src
   python -m app.main
```

Repositórios (dados)

O sistema usa repositório via injeção de dependência no service, permitindo trocar a implementação sem mudar a regra de negócio.

- Repositório CSV: persiste em `controle_estoque/data/produtos.csv` e não permite `codigo` duplicado.
- Repositório em memória: carrega do CSV na inicialização e opera somente em memória (não persiste ao reiniciar). Se detectar `codigo` duplicado ao carregar, levanta `ValueError` (conflito).

O contrato mínimo do repositório (para o service aceitar qualquer implementação) está em `repositories/protocols.py`.

```
### Como trocar o repositório

No arquivo `controle_estoque/src/app/main.py`, altere qual classe é instanciada:

- Em memória: `ProdutoInMemoryRepository()`
- CSV: `ProdutoRepository()`

## 📈 Melhorias Futuras

- Implementar feedback visual durante processamento
- Criar navegação interativa por teclado
- Implementar ordenação por critérios (nome, preço, quantidade)
- Melhorar tratamento de erros no console (exibir mensagem e continuar)
```

controle_estoque/pyproject.toml

```
[build-system]
requires = ["setuptools >= 61.0", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "controle-estoque"
version = "0.1.0"
```

controle_estoque/src/app/config.py

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent.parent
DATA_DIR = BASE_DIR / "data"
DATA_DIR.mkdir(exist_ok=True)  # garante que exista
```

controle_estoque/src/app/main.py

```
# Agora os imports funcionam
from .utils.console import carregamento_pontos, limpar_console
from .services.produtos import ProdutoService
from .repositories.memoria_produtos import ProdutoInMemoryRepository
from .models.produto import Produto

# MVP
""" Cadastrar Produto
    Permitir o cadastro de um novo produto no
    estoque, solicitando ao usuário as informações necessárias. O
    sistema não deve permitir o cadastro de produtos com código
```

```
    duplicado e deve validar os dados informados, garantindo que
    o preço e a quantidade não sejam negativos.
"""
""" Calcular o Total de Produtos em Estoque
    Calcular e exibir a
    quantidade total de produtos armazenados no estoque,
    considerando a soma das quantidades de todos os produtos
    cadastrados.
"""

""" To-do
- Fazer função de carregamento visual com pontos
- Menu ser selecionável via teclado setas para mover e espaço para confirmar
- Formatar os produtos em tabelas
- Se entrar código repetido solicitar novamente os dados ao usuário

 --- Listagem
- Listar por ordem.... tipo por nome, por preço, etc

"""

repo = ProdutoInMemoryRepository()
service = ProdutoService(repo)

menu_ativo = True
res_usuario = ''

# Funções de cadastro - mover para controller?
def estoque_atual(produtos : list[Produto]) -> int:
    total_estoque = 0

    for p in produtos:
        total_estoque +=  p.quantidade

    return total_estoque

def iniciar_cadastro():
    limpar_console()
    produto = {}
    print("----> Cadastro de novo produto iniciado <----")
    print("Insira as informações do produto")
    produto["codigo"] = input("Digite o código: ")
    produto["nome"] = input("Digite o nome: ")

    # Precisa inserir com ponto pra dar certo
    # adaptar para o usuário colocar virgula e funcionar
    preco = input("Digite o preço(ex: 10,50): ")
    # Slicing [start:stop:step]
    preco_invertido = preco[::-1]
    print("preço invertido", preco_invertido)
    preco_invertido = preco_invertido.replace(".", "")
    preco = preco_invertido.replace(",",".", 1)

    preco = preco[::-1]
```

```
    print("Preço verificado ->",preco)

    produto["preco"] = float(preco)
    produto["quantidade"] = int(input("Digite a quantidade em estoque: "))

    # Depois de tratar os inputs
    service.cadastrar(**produto)


def listagem_produtos():
    limpar_console()
    print("----> Produtos cadastrados <----")
    produtos = service.listar_todos()
    if not produtos:
        print("Não há produtos registrados")
    else:
        for prod in produtos:
            print(prod)

    estoque_total = estoque_atual(produtos)
    print("Estoque total:", estoque_total)
    print("\n")


def menu(menu_ativo):
    # To-do legal -> Ser selecionável via teclado setas para mover e espaço para
confirmar
    # Colocar comando para fechar o menu
    while (menu_ativo):
        print("------ Menu de acesso ------")
        # Ao ver produtos temos que ter uma funcionalidade de calcular quantos
prods tem em estoque
        print("- Ver produtos [1]")
        print("- Cadastrar produtos [2]")
        print("- Fechar menu digite -> Fechar\n")

        res_usuario = input("Digite a opção desejada: ")

        print(res_usuario)

        match res_usuario.lower():
            case "fechar":
                menu_ativo = False
                carregamento_pontos()
                print("Menu encerrado")

                break
            case "1":
                listagem_produtos()
            case "2":
                iniciar_cadastro()
            case _:
                limpar_console()
```

```
                print("Digite uma opção valida\n")


menu(menu_ativo)
```

controle_estoque/src/app/models/produto.py

```
from dataclasses import dataclass, field
import uuid
from datetime import date

# @dataclass(frozen=True) frozen para tornar a entidade imutável
@dataclass (frozen=True) # Não permite alterações na entidade
# Parâmetros com valor default dever vir após parâmetros obrigatórios, do
contrário dá erro
class Produto:
    codigo: str
    nome: str
    preco: float
    quantidade: int
    id: uuid.UUID = field(default_factory=uuid.uuid4) # o init=False diz que ao
criar um prod ele não precisa passar o argumento id, mas na hora de montar
novamente a entidade dá problema

    # __post_init__ opcional para validações extras após criação
    def __post_init__(self):
        if not self.codigo:
            raise ValueError("Código obrigatório")

    def __str__(self):
        return (
            f"{self.nome}\n"
            f"código: {self.codigo}\n"
            f"preço: {self.preco}\n"
            f"quantidade em estoque: {self.quantidade}\n"
        )
```

controle_estoque/src/app/repositories/protocols.py

```
from __future__ import annotations

from typing import Protocol

from ..models.produto import Produto


class ProdutoRepositoryProtocol(Protocol):
    def adicionar(self, produto: Produto) -> None:
        ...
```

```
    def listar_todos(self) -> list[Produto]:
        ...
```

controle_estoque/src/app/repositories/memoria_produtos.py

```
from __future__ import annotations

from collections.abc import Iterable

from ..models.produto import Produto
from .produtos import ProdutoRepository
from .protocols import ProdutoRepositoryProtocol


class ProdutoInMemoryRepository:
    def __init__(self, *, loader: ProdutoRepositoryProtocol | None = None):
        self._produtos: list[Produto] = []
        self._por_codigo: dict[str, Produto] = {}

        if loader is None:
            loader = ProdutoRepository()

        self._carregar(loader.listar_todos())

    def _carregar(self, produtos: Iterable[Produto]) -> None:
        for produto in produtos:
            if produto.codigo in self._por_codigo:
                raise ValueError(
                    f"Conflito ao carregar dados: código {produto.codigo}
duplicado."
                )
            self._por_codigo[produto.codigo] = produto
            self._produtos.append(produto)

    def adicionar(self, produto: Produto) -> None:
        if produto.codigo in self._por_codigo:
            raise ValueError(f"Código {produto.codigo} já existe!")

        self._por_codigo[produto.codigo] = produto
        self._produtos.append(produto)

    def listar_todos(self) -> list[Produto]:
        return list(self._produtos)
```

controle_estoque/src/app/repositories/produtos.py

```
import csv
from pathlib import Path
```

```
from ..models.produto import Produto
from ..utils.arquivos import arquivo_vazio
from ..config import DATA_DIR
import uuid

CAMPOS = ["codigo", "nome", "preco", "quantidade"]
# CSV_FILE = Path("src/data/produtos.csv")
CSV_FILE = DATA_DIR / "produtos.csv"


class ProdutoRepository:
    # Métodos dentro de classe, métodos de instancia
    def __init__(self):
        self._verificar_arquivo()

    # Métodos privados iniciam a com _, mas se trata de uma convenção não temos
como encapsular realmente
    def _verificar_arquivo(self):
        if not CSV_FILE.exists():
            # Cria pasta data se nào existir
            CSV_FILE.parent.mkdir(parents=True, exist_ok=True)
            with CSV_FILE.open("w", newline="", encoding="utf-8") as file:
                writer = csv.writer(file)  # Cria escritor CSV
                # Seria possível assim: nova_lista = ["id", *CAMPOS] com uso de
args
                writer.writerow(["id", *CAMPOS])

    # A seta é pra mostrar o tipo que vai ser retornado
    # Type hint
    def _ler_todos(self) -> list[dict]:
        if not CSV_FILE.exists():
            return []
        with CSV_FILE.open("r", encoding="utf-8") as file:
            reader = csv.DictReader(file)
            return list(reader)

    def _salvar_todos(self, produtos: list[dict]):
        # with ->
        with CSV_FILE.open("w", newline="", encoding="utf-8") as file:
            writer = csv.DictWriter(file, fieldnames=["id", *CAMPOS])
            writer.writeheader()
            writer.writerows(produtos)

    def adicionar(self, produto: Produto, ) -> None:
        produtos = self._ler_todos()
        if any(p["codigo"] == produto.codigo for p in produtos):
            raise ValueError( f"Código {produto.codigo} já existe!")

        print(produtos)
        produtos.append({
            "id": str(produto.id),
            "codigo": produto.codigo,
            "nome": produto.nome,
            "preco": str(produto.preco),
```

```
            "quantidade": str(produto.quantidade)
        })

        print(produtos)
        self._salvar_todos(produtos)

    def listar_todos(self) -> list[Produto]:
        dados = self._ler_todos()
        return [
            Produto(
                id=uuid.UUID(p["id"]),
                codigo=p["codigo"],
                nome=p["nome"],
                preco=float(p["preco"]),
                quantidade=int(p["quantidade"])
            )
            for p in dados
        ]
```

controle_estoque/src/app/services/produtos.py

```
from ..models.produto import Produto
from ..repositories.protocols import ProdutoRepositoryProtocol

class ProdutoService:
    # Injeção de dependência
    # Aqui estamos definindo que PessoaService espera receber um repository que
seja ou herde
    # de ProdutoRepository, dessa forma evita acoplamento inserindo o
ProdutoRepository
    # diretamente dentro da ProdutosService
    def __init__(self, repo: ProdutoRepositoryProtocol):
        # Armazena o repositório recebido como um atributo de instância
        # Dessa forma conseguimos chamar a qualquer momento dentro da
ProdutoService
        # self.repo.adicionar()
        self.repo = repo

    def cadastrar(self, codigo: str, nome: str, preco: float, quantidade: int) ->
Produto:
        # Validações

        # verificar se cod já existe
        # Posso criar uma busca por cod, se retornar False(Não encontrado), pode
continuar
        # Ou se encontrar, dá erro

        # verificar se o preço ou estoque não está negativo
        if preco < 0:
            raise ValueError("Preços negativos não são permitidos.")
```

```
        if quantidade < 0:
            raise ValueError("Quantidades negativas não são permitidas.")



        # Após verificação montar entidade
        novo_produto = Produto (
            codigo=codigo,
            nome= nome.strip().title(),
            preco=preco,
            quantidade=quantidade
        )
        print(novo_produto)

        self.repo.adicionar(novo_produto)
        return novo_produto

    def listar_todos(self) -> list[Produto]:
        return self.repo.listar_todos()
```

controle_estoque/src/app/utils/arquivos.py

```
import os

def arquivo_vazio(arquivo: str) -> bool:
    # Verifica se arquivo existe
    if not os.path.exists(arquivo):
        return True
    # Verifica se está vazio
    return os.path.getsize(arquivo) == 0
```

controle_estoque/src/app/utils/console.py

```
import os
import time

# Limpar console
def limpar_console():
    os.system("cls" if os.name == "nt" else "clear")


def carregamento_pontos():
    print("...")
    time.sleep(.5)
    print("\033[A                             \033[A")
    print("..")
    time.sleep(.5)
    print("\033[A                             \033[A")
```

```
        time.sleep(.5)
        print(".")
        time.sleep(.5)
        print("\033[A                                \033[A")
```